

Banking Virtual Assistant: Migration Options Analysis

Context: Current VA runs on Google Dialogflow CX with deterministic flows covering cards, accounts, transactions, money movement, spending insights, transfer-to-human, rates, fees, and limits.

Primary driver for change: Vendor lock-in and lack of flexibility on the current platform.

Team capability: Strong, dedicated in-house ML/LLM platform team.

Option 1: Full Migration to Internally Built LLM-Powered Conversational System

All deterministic Dialogflow flows are re-implemented as generative flows orchestrated by an internally built LLM system (e.g., agentic orchestration, tool/function calling into banking APIs).

Pros

- **Full control and no vendor lock-in** — architecture, model choice, and roadmap are entirely owned in-house; can swap underlying LLM providers as the market evolves.
- **Flexibility** — natural handling of compound, ambiguous, or multi-intent requests ("what's my balance and can you also dispute this charge") that rigid flows struggle with.
- **Faster iteration** — no dependency on Google's release cadence, quotas, or CX Designer UI constraints; internal team can ship changes on their own schedule.
- **Deeper integration potential** — can be embedded directly into the bank's existing data platform, risk engine, fraud checks, and case management tools without third-party SaaS boundaries.
- **Cost model control** — token/compute costs can be optimized and negotiated directly (model choice, caching, self-hosting options) rather than fixed per-conversation/per-request Google pricing.
- **Differentiation** — proprietary conversational experience can become a competitive asset rather than a commodity SaaS layer.

Cons

- **Highest execution risk and cost** — building an agentic system that reliably handles transactional actions (transfers, card actions, disputes) at banking-grade accuracy is a multi-quarter (often multi-year) investment even with a strong team.
- **Regulatory/compliance burden shifts in-house** — audit trails, explainability, model risk management (SR 11-7 / OCC model risk guidance equivalents), bias testing, and change control all become the bank's responsibility rather than a vendor's.

- **Hallucination and reliability risk on transactional flows** — money movement, limits, and account changes need near-zero error tolerance; LLM-driven execution requires heavy guardrails (constrained generation, deterministic execution layers behind the LLM, human-in-the-loop confirmation for high-risk actions).
 - **New operational burden** — need model monitoring, drift detection, prompt/version management, red-teaming, incident response, and 24/7 on-call ownership that Google previously handled.
 - **Migration risk** — cutting over dozens of existing flows simultaneously (or in a big-bang) risks regressions in a live, regulated customer channel.
 - **Talent dependency** — sustained reliance on retaining the specialized ML/LLM team; attrition risk is now a business continuity risk for a core banking channel.
-

Option 2: Remain on Google Dialogflow CX (Google's Agentic Conversational System)

Continue building/maintaining deterministic (and Google's newer agentic CX) flows on the existing Google platform.

Pros

- **Lowest near-term risk and cost** — no re-platforming effort; existing flows, integrations, and institutional knowledge remain valid.
- **Vendor-managed compliance surface** — Google maintains uptime SLAs, security certifications, and platform-level compliance (SOC2, etc.), reducing the bank's direct operational/compliance burden for the platform itself.
- **Predictable behavior** — deterministic flows are inherently more auditable and testable for regulators; behavior is fully specified rather than probabilistic.
- **Faster time-to-value for incremental changes** — well understood platform, existing tooling, existing staff proficiency.
- **CX's newer generative/agentic capabilities** can be adopted incrementally without a full re-architecture, if desired.

Cons

- **Vendor lock-in persists (the core stated driver for change)** — pricing, roadmap, data residency, and platform capabilities remain outside the bank's control.
- **Flexibility ceiling** — even Google's agentic CX layer is still bounded by Google's design choices, release cadence, and configuration model; deep customization for bank-specific workflows/data will always be constrained relative to a fully owned system.
- **Limited differentiation** — the conversational experience is inherently a commodity layer shared across all Dialogflow customers.

- **Cost growth exposure** — usage-based/licensing pricing scales with volume, with limited negotiating leverage as usage grows.
 - **Doesn't address the underlying problem** — the strategic goal of reducing lock-in and increasing flexibility isn't served by staying as-is; this is effectively deferring the decision rather than resolving it.
-

Option 3: Hybrid

Two distinct hybrid architectures are possible, and they carry different tradeoffs — worth evaluating as separate sub-options rather than one blended "hybrid" bucket.

3a. Flow-Based Hybrid — Selected Flows Migrate, Others Remain on Dialogflow CX

High-value or high-flexibility-need flows (e.g., complex/ambiguous queries, spending insights, cross-domain questions) migrate to the internal LLM system, while stable, well-defined, or highly regulated transactional flows (e.g., money movement, limits changes) remain fully on Dialogflow CX (both NLU/dialogue management and voice/avatar).

Pros

- **Risk-calibrated migration** — highest-compliance-sensitivity flows (transfers, account changes) stay on the more auditable deterministic platform while flexibility gains are captured where they matter most.
- **Incremental de-risking of vendor lock-in** — begins reducing dependency without betting the entire channel on an unproven internal system on day one.
- **Learning curve built in** — the internal team builds production trust on lower-risk flows before taking on transactional ones.
- **Natural rollback path** — an underperforming flow can stay on/return to Dialogflow without disrupting the whole assistant.

Cons

- **Two full conversational stacks to operate** — duplicated NLU, dialogue management, tooling, monitoring, and on-call across both platforms.
- **Inconsistent user experience risk** — tone, latency, and conversational memory can differ noticeably across a mid-conversation handoff (e.g., balance question → transfer request).
- **Handoff/context-passing complexity** — session state, entities, and authentication state must cross platform boundaries cleanly; a common source of dropped context.
- **Still partially exposed to vendor lock-in** — whatever remains on Dialogflow indefinitely leaves the original driver only partially addressed.
- **Ambiguous end state** — without sunset criteria, this can become a permanent, uncosted dual-stack.

3b. Capability-Based Hybrid — Internal LLM Owns Conversation Logic, Google Retained Only Where It Has a Clear Edge (Voice, Avatar, Multilingual, Interruption Handling)

Rather than buying the full Dialogflow CX platform, the bank builds the conversational brain (NLU, dialogue management, orchestration, banking-action execution) entirely in-house, and licenses only the narrower Google capabilities that are genuinely hard/expensive to replicate — e.g., speech-to-text/text-to-speech quality, multilingual coverage, real-time barge-in/interruption handling, and avatar rendering.

Pros

- **Single conversational brain** — one system owns intent, dialogue state, and business logic across every flow and channel (chat, voice, avatar), avoiding the double-implementation and routing problems of 3a.
- **Consistent experience everywhere** — tone, policy, and conversational memory are uniform regardless of modality, since Google is providing infrastructure, not decision-making.
- **De-risks the strategic asset, not the commodity layer** — conversation logic and business rules (the actual differentiator and the thing tied to lock-in concerns) move fully in-house, while genuinely hard, non-differentiating problems (ASR/TTS accuracy across languages, natural interruption handling, avatar rendering) stay with a vendor built to solve them at scale.
- **Likely lower ongoing licensing cost** than a full CX agent-building platform, since only narrower API surfaces (speech, avatar) are consumed rather than the whole SaaS product.
- **Cleaner reversibility** — swapping a speech/avatar vendor later is a narrower, more contained integration change than replacing an entire conversational platform.

Cons

- **Needs technical validation up front** — confirm whether Google's speech/avatar capabilities are usable standalone (via discrete APIs) independent of the CX agent-building layer, or whether they're bundled in ways that make decoupling harder than expected.
- **Real-time integration complexity** — the internal system must bridge to Google's streaming speech/avatar APIs with low enough latency to support natural turn-taking; interruption/barge-in handling specifically requires the internal system to detect and react to a signal fast enough to halt generation/TTS playback mid-utterance.
- **Two vendors to coordinate technically** (internal system + Google infra), even though only one owns the business logic — session management, auth, and error

handling need to be engineered across the boundary.

- **Still a form of vendor dependency**, just narrower and more modular — voice/avatar quality and pricing remain Google's to set, though switching cost is lower than full-platform lock-in.
 - **All transactional-risk considerations from Option 1 still apply**, since the internal system now owns 100% of the conversational and execution logic, including for money movement and card actions — this variant doesn't reduce transactional risk exposure the way 3a does by keeping some flows on deterministic rails.
-

3c. Blended Hybrid — Combine Both Axes

3a and 3b aren't mutually exclusive — they operate on two independent axes, and most real-world hybrid designs would combine both:

- **Axis 1 (per-flow platform decision):** which flows/journeys run fully on Dialogflow CX (as a complete SaaS product — NLU, dialogue management, and voice/avatar together) versus which run on the internal LLM system.
- **Axis 2 (per-capability sourcing decision):** *for the flows that do run on the internal system*, which underlying capabilities are still sourced from Google as discrete component services (speech-to-text/text-to-speech, avatar rendering, multilingual coverage, interruption handling) rather than built in-house — independent of whether the conversational logic itself is internal.

Put differently: 3a decides *which journeys* get Google's full product; 3b decides, *for the journeys that don't*, which of Google's individual capabilities are still worth buying rather than building. Applying both together is likely the realistic hybrid target state, e.g.:

- Simple, stable, highly regulated transactional flows (transfers, limit changes) → remain fully on Dialogflow CX as a complete product (Axis 1), for the reasons in 3a.
- Complex/ambiguous/cross-domain flows (spending insights, multi-intent requests, escalations) → migrate to the internal LLM system for conversation logic (Axis 1), while still using Google's speech, avatar, and multilingual APIs as the voice/visual delivery layer underneath that internal logic (Axis 2), for the reasons in 3b.

Additional considerations specific to this blended model

- **A third integration surface emerges**, beyond "internal LLM ↔ Dialogflow CX" (from 3a) and "internal LLM ↔ Google speech/avatar APIs" (from 3b): you now need consistent voice/avatar behavior *whether* a given turn is being driven by Dialogflow CX or by the internal LLM system, so the experience doesn't feel different depending on which flow a customer is in.
- **Governance question to resolve early:** what's the explicit rule for classifying a flow as "stays on full Dialogflow CX" vs. "migrates to internal LLM, sourcing Google only for

voice/avatar"? Without a clear, documented classification policy (e.g., tied to transaction risk tier, regulatory sensitivity, or conversational complexity), this blended model is the hardest of all four options to reason about, audit, and explain to regulators.

- **Vendor negotiation gets more complex, not less** — you'd be negotiating both a (smaller) Dialogflow CX agent-building license for retained flows *and* a separate/standalone speech-and-avatar API contract for the migrated flows, rather than one clean commercial relationship.
 - **This is the most flexible but also most operationally complex option overall** — it captures the benefits of 3a and 3b simultaneously, but inherits (and compounds) both of their integration/coordination costs rather than avoiding them.
-

Summary Comparison

Dimension	Full Internal Migration	Stay on Dialogflow CX	3a. Flow-Based Hybrid	3b. Capability-Based Hybrid
Addresses vendor lock-in	Fully	Not at all	Partially (only migrated flows)	Mostly — only voice/avatar infra remains external
Flexibility gain	Highest	Lowest	Medium-High (where migrated)	High, uniformly across all flows
Time to value	Longest	Immediate (status quo)	Medium (phased)	Medium-Long (still building full brain, but narrower vendor scope)
Upfront cost/effort	Highest	Lowest	Medium	Medium-High
Regulatory/compliance burden shift	Fully in-house	Stays with Google	Split, with added routing complexity	Fully in-house for conversation/logic; Google retains only infra-level compliance
Transactional risk exposure	Highest (until guardrails mature)	Lowest (proven deterministic)	Managed — transactions can stay on deterministic rails	Same as full migration — all logic (incl. transactions) is internal
Long-term operational complexity	Single stack, high initial build	Single stack, low build	Dual full stack, ongoing	Single conversational stack + narrower vendor integration (speech/avatar)
Reversibility	Low (heavy re-investment to revert)	N/A	High (flow-by-flow rollback possible)	Medium (swapping speech/avatar vendor is contained; swapping the conversational brain is not)

Key Considerations Specific to a Banking VA

- **Transactional vs. informational split matters most.** Given this VA handles money

movement, card actions, and limit changes, the accuracy/auditability bar for those specific flows is likely the single biggest factor in the decision — independent of which overall path is chosen.

- **Regulatory posture:** whichever option is chosen, expect scrutiny on explainability, audit logging of every decision path, and model risk management documentation for any LLM-driven action that changes account state.
 - **Guardrail pattern regardless of path:** a common approach is to let the LLM handle *understanding* (intent, entity extraction, disambiguation) while a deterministic, tightly scoped execution layer (not the LLM itself) actually performs the banking action — this pattern can be applied within Option 1 or Option 3 to reduce transactional risk.
 - **Sunset criteria for hybrid:** if Option 3 is pursued, it's worth defining upfront which flows are permanently deterministic (by policy, e.g., "any flow that moves money always stays on rails X") versus which are temporarily deterministic pending internal system maturity — otherwise hybrid risks becoming a permanent, uncoded dual-stack.
 - **Exit cost from Dialogflow:** worth separately scoping how much conversation history, training data, and flow logic is portable/reusable versus needing to be rebuilt from scratch, since this affects the real cost of Options 1 and 3.
-

Hedging Against GenAI Hype-Fade and LLM Cost-at-Volume Risk via Hybrid

Two distinct risks are worth separating, since hybrid protects against them in different ways: (1) the internal GenAI investment underdelivers or gets deprioritized, and (2) LLM inference cost at production conversation volume turns out to be economically unworkable.

Against investment/technology risk ("buzz fades")

The protection here is optionality, not technology. Full migration removes the working fallback — if the internal system underdelivers or the roadmap slips, there's no functioning deterministic system left to fall back on. Hybrid keeps Dialogflow CX running the flows that weren't migrated, so a stalled or disappointing GenAI investment doesn't break the channel — it simply caps how far the internal footprint expands. The exit ramp that doesn't exist in Option 1 is the actual insurance value of hybrid.

Against unit-economics risk (LLM cost at volume)

Hybrid can do more than reduce risk here — it can actively route around the cost problem, if flows are classified partly by **volume** rather than only by regulatory risk tier. In most banking VAs, a small number of flow types (balance checks, simple FAQs, rates/fees/limits lookups) account for the bulk of conversation volume — and these are also the flows where an LLM adds the least incremental value, since deterministic templates already serve them

well. Keeping high-volume, low-complexity flows on Dialogflow CX means the bulk of traffic never touches per-token LLM cost at all. Reserve the LLM for the lower-volume, higher-complexity long tail (ambiguous, multi-intent, cross-domain queries), where flexibility is actually earning its cost — a much smaller cost base to absorb even if per-conversation LLM cost runs high.

Suggested flow classification: volume × LLM value-add (in addition to risk tier)

	Low LLM value-add	High LLM value-add
High volume	Stay on Dialogflow CX — cost hedge; deterministic templates already serve these well	Watch closely — largest exposure if LLM unit costs balloon; strongest candidate for tiered/cheaper models or aggressive caching
Low volume	Lowest priority to touch either way	Migrate to internal LLM — flexibility pays off, and the cost base is small enough to absorb even at higher per-conversation cost

This is a second, independent axis on top of the risk-tier classification already used to decide which flows migrate (Option 3a) — in practice, both risk tier and volume/value-add should inform the same per-flow decision.

The honest caveat: hybrid isn't free insurance

Running two stacks carries a fixed operational tax — duplicated tooling, duplicated on-call, routing/handoff logic — regardless of whether either risk (hype-fade or cost blowup) ever materializes. If the internal GenAI investment pans out and costs stay reasonable, that duplication cost was paid for a hedge that wasn't needed. This is asymmetric protection: cheap relative to the downside it covers, but not zero-cost, and worth being explicit about when presenting hybrid as a "safer" option — it's safer, not free.

Cost-control levers that reduce this risk further, independent of which option is chosen

Within whatever portion of the assistant ends up LLM-owned (whether via hybrid or full migration), these reduce exposure to the cost-at-volume risk regardless of overall architecture choice:

- **Tiered model routing** — cheap/fast models for intent classification and simple turns; frontier models reserved for genuinely ambiguous or high-complexity cases.
- **Deterministic execution layer behind the LLM** — the LLM handles understanding (intent/entity extraction), not full response generation on every turn, with a scoped execution layer actually performing the banking action.
- **Caching and context minimization** — reduces repeated token cost for common conversational patterns and repeated context (e.g., account state, prior turns) sent per

request.

Open Questions Worth Resolving Before Deciding

1. What is the acceptable error rate / risk tolerance for LLM-initiated transactional actions (transfers, limit changes) versus informational responses?
2. Is there a target timeline or regulatory deadline driving urgency (e.g., existing Google contract renewal date)?
3. Would the internal system be built on a self-hosted/open-weight model, a third-party API (further vendor dependency, just a different vendor), or a mix?
4. What existing infrastructure does the ML/LLM platform team have in production today (if any) that's directly reusable here (e.g., orchestration frameworks, evaluation pipelines, guardrail tooling)?
5. Is there an existing case management / human-handoff system that both paths would need to integrate with equally?